

Common Tricks

常见技巧

本页面主要列举一些竞赛中的小技巧。

利用局部性

局部性是指程序倾向于引用邻近于其他最近引用过的数据项的数据项，或者最近引用过的数据项本身。局部性分为时间局部性和空间局部性。

具体可参见 [循环展开 \(Loop Unroll\)](#)、[代码布局优化 \(Code Layout Optimizations\)](#) 等内容

循环宏定义

如下代码可使用宏定义简化：

```
1 for (int i = 0; i < N; i++) {
2     // 循环内容略
3 }
4
5 // 使用宏简化
6 #define f(x, y, z) for (int x = (y), __ = (z); x < __; ++x)
7
8 // 这样写循环代码时，就可以简化成 `f(i, 0, N)`。例如：
9 // a is a STL container
10 f(i, 0, a.size()) { ... }
```

另外推荐一个比较有用的宏定义：

```
1 #define _rep(i, a, b) for (int i = (a); i <= (b); ++i)
```

善用 namespace

使用 namespace 能使程序可读性更好，便于调试。

► 例题：NOI 2018 屠龙勇士

使用宏进行调试

编程者在本地测试的时候，往往要加入一些调试语句。而在需要提交到 OJ 时，为了不使调试语句的输出影响到系统对程序输出结果的判断，就要把它们全部删除，耗时较多。这种情况下，可以通过定义宏的方式来节省时间。大致的程序框架是这样的：

```
1 #define DEBUG
2 #ifdef DEBUG
3 // do something when DEBUG is defined
4 #endif
5 // or
6 #ifndef DEBUG
7 // do something when DEBUG isn't defined
8 #endif
```

`#ifdef` 会检查程序中是否有 `#define` 定义的对应该标识符，如果有定义，就会执行后面的语句。而 `#ifndef` 会在没有定义相应标识符的情况下执行后面的语句。

这样，只需在 `#ifdef DEBUG` 里写好调试用代码，`#ifndef DEBUG` 里写好真正提交的代码，就能方便地进行本地测试。提交程序的时候，只需要将 `#define DEBUG` 一行注释掉即可。也可以不在程序中定义标识符，而是通过 `-DDEBUG` 的编译选项在编译的时候定义 `DEBUG` 标识符。这样就可以在提交的时候不用修改程序了。

不少 OJ 都开启了 `-DONLINE_JUDGE` 这一编译选项，善用这一特性可以节约不少时间。

对拍

对拍是一种进行检验或调试的方法，通过对比两个程序的输出来检验程序的正确性。可以将自己程序的输出与其他程序的输出进行对比，从而判断自己的程序是否正确。

对拍过程要多次进行，因此需要通过批处理的方法来实现对拍的自动化。

具体而言，对拍需要一个 [数据生成器](#) 和两个要进行输出结果比对的程序。

每运行一次数据生成器都将生成的数据写入输入文件，通过重定向的方法使两个程序读入数据，并将输出写入指定文件，最后利用 Windows 下的 `fc` 命令比对文件（Linux 下为 `diff` 命令）来检验程序的正确性。如果发现程序出错，可以直接利用刚刚生成的数据进行调试。

对拍程序的大致框架如下：

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     // For Windows
6     // 对拍时不开文件输入输出
7     // 当然，这段程序也可以改写成批处理的形式
8     while (true) {
9         system("gen > test.in"); // 数据生成器将生成数据写入输入文件
10        system("test1.exe < test.in > a.out"); // 获取程序1输出
11        system("test2.exe < test.in > b.out"); // 获取程序2输出
12        if (system("fc a.out b.out")) {
13            // 该行语句比对输入输出
14            // fc返回0时表示输出一致，否则表示有不同处
15            system("pause"); // 方便查看不同处
16            return 0;
17            // 该输入数据已经存放在test.in文件中，可以直接利用进行调试
18        }
19    }
20 }
```

内存池

当动态分配内存时，频繁使用 `new/malloc` 会占用大量的时间和空间，甚至生成大量的内存碎片从而降低程序的性能，可能会使原本正确的程序 TLE/MLE。

这时候需要使用到「内存池」这种技巧：在真正使用内存之前，先申请分配一定大小的内存作为备用。当需要动态分配时直接从备用内存中分配一块即可。

在大多数 OI 题当中，可以预先算出需要使用到的最大内存并一次性申请分配。

示例：

```
1 // 申请动态分配 32 位有符号整数数组：
2 int* newarr(int sz) {
3     static int pool[MAXN], *allocp = pool;
4     return allocp += sz, allocp - sz;
5 }
6
7 // 线段树动态开点的代码：
8 Node* newnode() {
9     static Node pool[MAXN << 1], *allocp = pool - 1;
10    return ++allocp;
11 }
```

参考资料

[洛谷日报 #86](#)

《算法竞赛入门经典 习题与解答》

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[StudyingFather](#), [H-J-Granger](#), [Marcythm](#), [billchenchina](#), [ChungZH](#), [greyqz](#), [lr1d](#), [NachtgeistW](#), [partychicken](#), [Psycho7](#), [Xeonacid](#), [CCXXXI](#), [countercurrent-time](#), [Enter-tainer](#), [ksyx](#), [SukkaW](#), [Suyun514](#), [Suyun514](#), [Tiphereth-A](#), [TrisolarisHD](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用